

A Comparative Study of MLP and CNN Neural Tangent Kernels: Infinite-Width Limits and Finite-Width Behavior

Desbois-Renaudin Méo
meo.desbois@gmail.com

June 7, 2026

Abstract

Neural tangent kernels provide a way to study the training dynamics of wide neural networks through deterministic kernel limits. While the NTK of fully-connected networks is well understood, convolutional architectures encode locality and weight sharing, which induce different kernel geometries. In this work, we recall the standard NTK recursion for multilayer perceptrons and derive the covariance and NTK recursions for a simple convolutional architecture with global average pooling. We then compare finite-width MLPs and CNNs on MNIST through kernel convergence, NTK drift during training, translation behavior, spectral properties of Gram matrices, and class-sorted kernel similarities. The experiments show that architecture has a strong influence on the induced NTK: CNN kernels better preserve similarity under small translations, while spectral and class-based analyses reveal different organizations of the data before training. This study is intended as a controlled and pedagogical comparison of MLP and CNN NTKs, rather than as a new theoretical framework.

1 Introduction

Deep neural networks achieve strong empirical performance, but their training dynamics remain difficult to analyze directly. A useful approach is to study neural networks in the infinite-width limit, where their behavior can often be described through kernel methods. In particular, the neural tangent kernel (NTK), introduced by [JGH20] provides a description of gradient-based training in terms of a kernel that becomes deterministic at infinite width.

The NTK framework is well understood for fully-connected neural networks. In this setting, multilayer perceptrons converge, under suitable scaling, to deterministic covariance and NTK recursions. However, different architectures induce different kernels. This is especially important for image data: while an MLP treats an image as a flattened vector, a convolutional neural network encodes locality and weight sharing. These architectural choices change the geometry induced by the corresponding kernel.

The goal of this work is to compare MLP and CNN neural tangent kernels in a controlled setting. We focus on both the infinite-width kernels and the behavior of finite-width networks. The purpose is not to introduce a new theoretical framework, but to give a self-contained derivation for a simple convolutional architecture and to study how the resulting kernel differs from the MLP kernel on MNIST.

The main contributions of this work are:

- (i) we derive the covariance and NTK recursions for a simple convolutional architecture with global average pooling;
- (ii) we compare finite-width empirical kernels with their infinite-width limits;
- (iii) we study NTK drift during training as a measure of deviation from the lazy-training regime;
- (iv) we compare the kernel geometries induced by MLPs and CNNs through translation behavior, spectral properties, and class-sorted Gram matrices.

The experiments show that architecture has a strong influence on the NTK geometry. The CNN kernel better preserves similarity under small translations, reflecting the locality and weight sharing of convolutional layers. At the same time, spectral and class-based analyses show that the differences between MLP and CNN kernels are more nuanced than a simple statement that one architecture is uniformly better.

This work should therefore be read as a controlled and pedagogical comparison. All experiments are limited to MNIST and to finite subsets of the data because empirical NTK and Gram matrix computations are expensive. Nevertheless, the results illustrate how architectural choices are already visible at the level of the neural tangent kernel, before and during training.

2 Background: NN-GP and NTK Theory

This section recalls the two kernel limits used throughout the paper: the Neural Network Gaussian Process (NN-GP), which describes the distribution of infinitely wide networks at initialization, and the Neural Tangent Kernel (NTK), which describes their training dynamics in the lazy regime.

2.1 Infinite-width neural networks and Gaussian processes

Let $f(\cdot; \theta) : \mathbb{R}^{n_0} \rightarrow \mathbb{R}$ be a scalar-output neural network with parameters θ . At random initialization, sufficiently wide neural networks can often be described by a Gaussian process in the infinite-width limit. This idea goes back to the Bayesian interpretation of neural networks [Nea96], and was later formalized for deep networks through layerwise covariance recursions [LBN⁺18, MRH⁺18].

For inputs $x, x' \in \mathbb{R}^{n_0}$, the corresponding covariance kernel is

$$\Sigma(x, x') = \mathbb{E}_\theta [f(x; \theta)f(x'; \theta)].$$

In the infinite-width limit, this kernel becomes deterministic and can be computed recursively from one layer to the next. It therefore provides a first way of comparing architectures at initialization: different architectural choices induce different Gaussian-process priors over functions.

In this work, we focus on scalar-output networks. For multi-output classification networks, the kernels can be interpreted coordinate-wise, assuming independent and identically distributed output coordinates.

2.2 Neural Tangent Kernel

While the NN-GP kernel describes the network at initialization, it does not by itself describe how the network evolves during training. The Neural Tangent Kernel, introduced by Jacot, Gabriel

and Hongler [JGH20], captures the first-order sensitivity of the network output with respect to its parameters. For a scalar-output network, it is defined as

$$\Theta_{\theta}(x, x') = \nabla_{\theta} f(x; \theta)^{\top} \nabla_{\theta} f(x'; \theta).$$

Equivalently, $\Theta_{\theta}(x, x')$ measures how much an infinitesimal parameter update caused by the input x' changes the prediction at x .

In the infinite-width limit, under standard NTK parameterization, the empirical NTK converges to a deterministic kernel Θ_{∞} . Moreover, during gradient-flow training, this kernel remains approximately constant in the infinite-width limit [JGH20, LXS⁺19]. The network dynamics can then be approximated by kernel gradient descent with respect to the initial NTK.

2.3 Training dynamics in the NTK regime

Let $X = (x_1, \dots, x_n)$ be a training set. The Gram matrix associated with the NTK is

$$\Theta_{\theta}(X, X) = (\Theta_{\theta}(x_i, x_j))_{1 \leq i, j \leq n}.$$

For a loss $\mathcal{L}$ depending on the predictions $f_t(X)$, gradient-flow training satisfies

$$\partial_t f_t(X) = -\Theta_{\theta_t}(X, X) \nabla_f \mathcal{L}(f_t(X)).$$

In the NTK regime, the empirical kernel Θ_{θ_t} stays close to its initialization value Θ_{θ_0} , so the non-linear training dynamics are approximated by a linear kernel method.

This point of view is closely related to lazy training, where the parameters move only slightly from initialization and the network behaves approximately like its first-order Taylor expansion [COB19]. It provides a useful theoretical baseline, but it does not capture all finite-width effects or feature-learning behavior. For this reason, comparing empirical finite-width kernels with their infinite-width limits is an important way to quantify deviations from the lazy regime.

In the following sections, we derive and compare the NN-GP and NTK limits for fully-connected and convolutional architectures. The main question is how architectural structure, especially locality and weight sharing in CNNs, changes the induced kernels and therefore the associated training dynamics.

2.4 Notation and conventions

Throughout the paper, we use x and $\bar{x}$ to denote two inputs. Layer indices are denoted by l , neuron or channel indices by i, j , and the width of a hidden layer, or number of channels in a convolutional layer, by m . The activation function is denoted by ϕ .

For a neural network $f(\cdot; \theta)$, we denote by

$$\Sigma^{(l)}(x, \bar{x})$$

the covariance kernel at layer l , and by

$$\Theta^{(l)}(x, \bar{x})$$

the neural tangent kernel accumulated up to layer l . When needed, the dependence on the parameters is written explicitly as $\Theta_{\theta}^{(l)}$. Otherwise, $\Theta^{(l)}$ refers to the infinite-width deterministic limit.

For convolutional networks, spatial positions are denoted by

$$\alpha, \bar{\alpha} \in \Omega^{(l)},$$

where $\Omega^{(l)}$ is the spatial grid at layer l . In this case, the covariance and NTK may also depend on spatial coordinates:

$$\Sigma^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}), \quad \Theta^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}).$$

The precise convolutional architecture, including patches, channels, filters, and readout, is introduced in Section 4.

Given a dataset $X = (x_1, \dots, x_n)$, any scalar kernel K induces a Gram matrix

$$K(X, X) = (K(x_a, x_b))_{1 \leq a, b \leq n}.$$

These Gram matrices are the main objects used in the experiments to compare MLP and CNN kernels, as well as finite-width empirical NTKs and their infinite-width limits.

3 MLP Baseline: Known NTK Recursion

3.1 MLP architecture

We consider a fully-connected network of depth L . For $l = 1, \dots, L$, the pre-activations and activations are defined by

$$h^{(l)}(x) = W^{(l)}x^{(l-1)}(x) + b^{(l)}, \quad x^{(l)}(x) = \phi(h^{(l)}(x)),$$

with $x^{(0)}(x) = x$.

We use the equivalent variance-scaled parameterization

$$W_{ij}^{(l)} \sim \mathcal{N}\left(0, \frac{\sigma_w^2}{n_{l-1}}\right), \quad b_i^{(l)} \sim \mathcal{N}(0, \sigma_b^2),$$

With this convention, the factor $1/\sqrt{n_{l-1}}$ is absorbed into the variance of the weights.

3.2 Covariance recursion

The infinite-width covariance recursion for MLPs is given by

$$\begin{aligned} \Sigma^{(1)}(x, \bar{x}) &= \frac{\sigma_w^2}{n_0} x^T \bar{x} + \sigma_b^2 \\ \Sigma^{(l+1)}(x, \bar{x}) &= \sigma_w^2 \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Lambda^{(l)})} [\phi(u)\phi(v)] + \sigma_b^2, \end{aligned}$$

where

$$\Lambda^{(l)} = \begin{pmatrix} \Sigma^{(l)}(x, x) & \Sigma^{(l)}(x, \bar{x}) \\ \Sigma^{(l)}(x, \bar{x}) & \Sigma^{(l)}(\bar{x}, \bar{x}) \end{pmatrix}.$$

3.3 MLP NTK recursion

As [JGH20] showed, the corresponding infinite-width NTK recursion can be written as

$$\begin{aligned}\Theta^{(1)}(x, \bar{x}) &= \Sigma^{(1)}(x, \bar{x}) \\ \Theta^{(l+1)}(x, \bar{x}) &= \Theta^{(l)}(x, \bar{x}) \dot{\Sigma}^{(l+1)}(x, \bar{x}) + \Sigma^{(l+1)}(x, \bar{x}),\end{aligned}$$

where

$$\dot{\Sigma}^{(l+1)}(x, \bar{x}) = \sigma_w^2 \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Lambda^{(l)})} [\phi'(u) \phi'(v)].$$

4 CNN Kernel Derivation

4.1 Convolutional operation

Let $\Omega \subset \mathbb{Z}^2$ be a spatial grid on an image with C channels. Throughout this paper, we will note the value of the i -th channel at position $\alpha \in \Omega$ of an image a as $a_i(\alpha)$.

Let $x \in \mathbb{R}^{C \times |\Omega|}$ be a input tensor and $W \in \mathbb{R}^{C' \times C \times |\mathcal{P}|}$ be a convolution kernel with $\mathcal{P} = \{0, 1, \dots, k-1\}^2$ the 2D patch for kernel size k . We will use the convolution operator without padding, defined on $\mathbb{R}^{C' \times C \times |\mathcal{P}|} \times \mathbb{R}^{C \times |\Omega|} \rightarrow \mathbb{R}^{C' \times |\Omega'|}$ following this behavior:

$$(W * x)_{c'}(\alpha) = \sum_{c=1}^C \sum_{\beta \in \mathcal{P}} W_{c',c}(\beta) x_c(\alpha + \beta)$$

for all $c' \in \{1, \dots, C'\}$ and all spatial positions $\alpha \in \Omega' := \{\alpha \in \Omega \mid \alpha + \mathcal{P} \subset \Omega\}$. In the case $\Omega = \{1, \dots, h\} \times \{1, \dots, w\}$, this corresponds to $\Omega' = \{1, \dots, h-k+1\} \times \{1, \dots, w-k+1\}$.

4.2 CNN architecture setup

We consider a convolutional neural network with L convolutional layers. For an input x , let $h^{(l)}(x)$ denote the pre-activation function evaluated on input x at layer l .

For each convolutional layer $l = 1, \dots, L$, we denote by $C^{(l)}$ the number of channels and by $\Omega^{(l)}$ the spatial domain. We define the pre-activations $h^{(l)}(x; \theta) \in \mathbb{R}^{C^{(l)} \times |\Omega^{(l)}|}$ and post-activations $x^{(l)}(x; \theta) \in \mathbb{R}^{C^{(l)} \times |\Omega^{(l)}|}$ via:

$$\begin{aligned}h^{(l+1)}(x; \theta) &= W^{(l+1)} * x^{(l)}(x; \theta) + b^{(l+1)}, \\ x^{(l+1)}(x; \theta) &= \phi \left(h^{(l+1)}(x; \theta) \right).\end{aligned}$$

The readin and readout are defined simply as the input and a Global average pooling layer.

$$\begin{aligned}x^{(0)}(x; \theta) &= x. \\ f(x; \theta) &= \frac{1}{|\Omega^{(L)}|} \sum_{\alpha \in \Omega} h^{(L)}(x; \theta)(\alpha)\end{aligned}$$

Finally, the weights are initialized with zero mean and variance independently across all indices and layers.

$$W_{c',c}^{(l)}(\beta) \sim \mathcal{N}\left(0, \frac{\sigma_w^2}{C^{(l-1)}|\mathcal{P}|}\right), \quad b_c^{(l)} \sim \mathcal{N}(0, \sigma_b^2),$$

4.3 Covariance recursion proof

We define the NNGP pre-activation covariance between two inputs $x, \bar{x}$, respectively evaluated at spacial coordinates $\alpha, \bar{\alpha}$ for layer l as

$$\Sigma^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) := \mathbb{E} \left[h_c^{(l)}(x; \theta)(\alpha) h_c^{(l)}(\bar{x}; \theta)(\bar{\alpha}) \right], \quad \alpha, \bar{\alpha} \in \Omega^{(l)}.$$

We will prove by induction in this section the following recursion. For $l = 1$, the covariance is initialized from the deterministic input image x :

$$\Sigma^{(1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sigma_b^2 + \frac{\sigma_w^2}{C^{(0)}|\mathcal{P}|} \sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} x_c(\alpha + \beta) \bar{x}_c(\bar{\alpha} + \beta).$$

For $l \geq 1$, the next layer covariance is computed recursively from layer l :

$$\begin{aligned} \lambda^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \begin{pmatrix} \Sigma^{(l)}(x, x; \alpha, \alpha) & \Sigma^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) \\ \Sigma^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) & \Sigma^{(l)}(\bar{x}, \bar{x}; \bar{\alpha}, \bar{\alpha}) \end{pmatrix} \\ \Sigma^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \sigma_b^2 + \frac{\sigma_w^2}{|\mathcal{P}|} \sum_{\beta \in \mathcal{P}} \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \lambda^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta))} [\phi(u)\phi(v)] \end{aligned}$$

We will first derive the expression of $\Sigma^{(1)}$, then prove the recursion by induction over l .

4.3.1 Initialization

For the first layer, the input $x^{(0)}(x; \theta) = x$ is deterministic. Hence, the only randomness in the first pre-activation comes from the weights and biases of the first convolutional layer.

By definition, for $c' \in \{1, \dots, C^{(1)}\}$ and $\alpha \in \Omega^{(1)}$, we have

$$h_{c'}^{(1)}(x; \theta)(\alpha) = \sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} W_{c',c}^{(1)}(\beta) x_c(\alpha + \beta) + b_{c'}^{(1)}.$$

Similarly, for another input image $\bar{x}$ and another spatial position $\bar{\alpha} \in \Omega^{(1)}$,

$$h_{c'}^{(1)}(\bar{x}; \theta)(\bar{\alpha}) = \sum_{\bar{c}=1}^{C^{(0)}} \sum_{\bar{\beta} \in \mathcal{P}} W_{c',\bar{c}}^{(1)}(\bar{\beta}) \bar{x}_{\bar{c}}(\bar{\alpha} + \bar{\beta}) + b_{c'}^{(1)}.$$

The first-layer covariance is therefore

$$\Sigma^{(1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \mathbb{E} \left[h_{c'}^{(1)}(x; \theta)(\alpha) h_{c'}^{(1)}(\bar{x}; \theta)(\bar{\alpha}) \right].$$

Substituting the expressions of the pre-activations gives

$$\begin{aligned} \Sigma^{(1)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \mathbb{E} \left[\left(\sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} W_{c',c}^{(1)}(\beta) x_c(\alpha + \beta) + b_{c'}^{(1)} \right) \right. \\ &\quad \left. \left(\sum_{\bar{c}=1}^{C^{(0)}} \sum_{\bar{\beta} \in \mathcal{P}} W_{c',\bar{c}}^{(1)}(\bar{\beta}) \bar{x}_{\bar{c}}(\bar{\alpha} + \bar{\beta}) + b_{c'}^{(1)} \right) \right]. \end{aligned}$$

Expanding the product, we obtain

$$\begin{aligned}
\Sigma^{(1)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \sum_{c, \bar{c}=1}^{C^{(0)}} \sum_{\beta, \bar{\beta} \in \mathcal{P}} x_c(\alpha + \beta) \bar{x}_{\bar{c}}(\bar{\alpha} + \bar{\beta}) \mathbb{E} \left[W_{c',c}^{(1)}(\beta) W_{c',\bar{c}}^{(1)}(\bar{\beta}) \right] \\
&\quad + \sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} x_c(\alpha + \beta) \mathbb{E} \left[W_{c',c}^{(1)}(\beta) b_{c'}^{(1)} \right] \\
&\quad + \sum_{\bar{c}=1}^{C^{(0)}} \sum_{\bar{\beta} \in \mathcal{P}} \bar{x}_{\bar{c}}(\bar{\alpha} + \bar{\beta}) \mathbb{E} \left[W_{c',\bar{c}}^{(1)}(\bar{\beta}) b_{c'}^{(1)} \right] \\
&\quad + \mathbb{E} \left[\left(b_{c'}^{(1)} \right)^2 \right].
\end{aligned}$$

Since the weights and biases are independent and have zero mean, the two weight-bias cross terms vanish:

$$\mathbb{E} \left[W_{c',c}^{(1)}(\beta) b_{c'}^{(1)} \right] = \mathbb{E} \left[W_{c',c}^{(1)}(\beta) \right] \mathbb{E} \left[b_{c'}^{(1)} \right] = 0.$$

Moreover, by independence of the weights across input channels and patch positions,

$$\mathbb{E} \left[W_{c',c}^{(1)}(\beta) W_{c',\bar{c}}^{(1)}(\bar{\beta}) \right] = \frac{\sigma_w^2}{C^{(0)}|\mathcal{P}|} \mathbf{1}_{c=\bar{c}} \mathbf{1}_{\beta=\bar{\beta}}.$$

The bias contribution is

$$\mathbb{E} \left[\left(b_{c'}^{(1)} \right)^2 \right] = \sigma_b^2.$$

Therefore, only the terms with $c = \bar{c}$ and $\beta = \bar{\beta}$ remain in the weight-weight sum. Hence,

$$\Sigma^{(1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \frac{\sigma_w^2}{C^{(0)}|\mathcal{P}|} \sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} x_c(\alpha + \beta) \bar{x}_c(\bar{\alpha} + \beta) + \sigma_b^2.$$

Thus, the first-layer covariance is

$$\boxed{\Sigma^{(1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sigma_b^2 + \frac{\sigma_w^2}{C^{(0)}|\mathcal{P}|} \sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} x_c(\alpha + \beta) \bar{x}_c(\bar{\alpha} + \beta)}$$

for all $\alpha, \bar{\alpha} \in \Omega^{(1)}$.

4.3.2 Induction step

Assume that, for some $l \geq 1$, the pre-activations at layer l converge in distribution to a centered Gaussian process with covariance

$$\Sigma^{(l)}(x, \bar{x}; \gamma, \bar{\gamma}) = \mathbb{E} \left[h_c^{(l)}(x; \theta)(\gamma) h_c^{(l)}(\bar{x}; \theta)(\bar{\gamma}) \right].$$

For $\gamma, \bar{\gamma} \in \Omega^{(l)}$, define

$$\lambda^{(l)}(x, \bar{x}; \gamma, \bar{\gamma}) = \begin{pmatrix} \Sigma^{(l)}(x, x; \gamma, \gamma) & \Sigma^{(l)}(x, \bar{x}; \gamma, \bar{\gamma}) \\ \Sigma^{(l)}(x, \bar{x}; \gamma, \bar{\gamma}) & \Sigma^{(l)}(\bar{x}, \bar{x}; \bar{\gamma}, \bar{\gamma}) \end{pmatrix}.$$

Thus, when $C^{(1)}, \dots, C^{(l-1)} \rightarrow \infty$,

$$\left(h_c^{(l)}(x; \theta)(\gamma), h_c^{(l)}(\bar{x}; \theta)(\bar{\gamma})\right) \sim \mathcal{N}\left(0, \lambda^{(l)}(x, \bar{x}; \gamma, \bar{\gamma})\right)$$

in the infinite-width limit.

Let $\alpha, \bar{\alpha} \in \Omega^{(l+1)}$. Since

$$h_{c'}^{(l+1)}(x; \theta)(\alpha) = \sum_{c=1}^{C^{(l)}} \sum_{\beta \in \mathcal{P}} W_{c',c}^{(l+1)}(\beta) \phi\left(h_c^{(l)}(x; \theta)(\alpha + \beta)\right) + b_{c'}^{(l+1)},$$

we have

$$\begin{aligned} \Sigma^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \mathbb{E} \left[h_{c'}^{(l+1)}(x; \theta)(\alpha) h_{c'}^{(l+1)}(\bar{x}; \theta)(\bar{\alpha}) \right] \\ &= \sigma_b^2 + \sum_{c, \bar{c}=1}^{C^{(l)}} \sum_{\beta, \bar{\beta} \in \mathcal{P}} \mathbb{E} \left[W_{c',c}^{(l+1)}(\beta) W_{c',\bar{c}}^{(l+1)}(\bar{\beta}) \right] \\ &\quad \times \mathbb{E} \left[\phi\left(h_c^{(l)}(x; \theta)(\alpha + \beta)\right) \phi\left(h_{\bar{c}}^{(l)}(\bar{x}; \theta)(\bar{\alpha} + \bar{\beta})\right) \right]. \end{aligned}$$

Using the independence and variance of the weights,

$$\mathbb{E} \left[W_{c',c}^{(l+1)}(\beta) W_{c',\bar{c}}^{(l+1)}(\bar{\beta}) \right] = \frac{\sigma_w^2}{C^{(l)} |\mathcal{P}|} \mathbf{1}_{c=\bar{c}} \mathbf{1}_{\beta=\bar{\beta}},$$

we obtain

$$\Sigma^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sigma_b^2 + \frac{\sigma_w^2}{C^{(l)} |\mathcal{P}|} \sum_{c=1}^{C^{(l)}} \sum_{\beta \in \mathcal{P}} \mathbb{E} \left[\phi\left(h_c^{(l)}(x; \theta)(\alpha + \beta)\right) \phi\left(h_c^{(l)}(\bar{x}; \theta)(\bar{\alpha} + \beta)\right) \right].$$

Since the channels are identically distributed, the expectation does not depend on c , and the sum over channels cancels the factor $1/C^{(l)}$. Therefore,

$$\Sigma^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sigma_b^2 + \frac{\sigma_w^2}{|\mathcal{P}|} \sum_{\beta \in \mathcal{P}} \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \lambda^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta))} [\phi(u) \phi(v)]$$

This proves the recursion.

4.3.3 Final readout covariance

We finally consider the global average pooling readout applied to the final pre-activations. The network output is defined by

$$f(x; \theta) = \frac{1}{|\Omega^{(L)}|} \sum_{\alpha \in \Omega^{(L)}} h^{(L)}(x; \theta)(\alpha).$$

Equivalently, for each output channel $c \in \{1, \dots, C^{(L)}\}$,

$$f_c(x; \theta) = \frac{1}{|\Omega^{(L)}|} \sum_{\alpha \in \Omega^{(L)}} h_c^{(L)}(x; \theta)(\alpha).$$

Since $f_c(x; \theta)$ is a linear average of Gaussian pre-activations in the infinite-width limit, it is also Gaussian. Therefore, the limiting network output is a Gaussian process whose covariance is obtained by averaging the final-layer pre-activation covariance over all pairs of spatial positions:

$$K(x, \bar{x}) = \mathbb{E} [f_c(x; \theta) f_c(\bar{x}; \theta)]$$

$$K(x, \bar{x}) = \frac{1}{|\Omega^{(L)}|^2} \sum_{\alpha, \bar{\alpha} \in \Omega^{(L)}} \Sigma^{(L)}(x, \bar{x}; \alpha, \bar{\alpha}).$$

Thus, the final Gaussian process kernel is obtained by spatially averaging the covariance $\Sigma^{(L)}$ of the last pre-activation layer.

4.4 CNN NTK recursion

We define the layerwise CNN NTK on pre-activations by

$$\Theta^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sum_{\theta \in \mathcal{D}_{\theta, \leq l}} \frac{\partial h_i^{(l)}(\alpha)}{\partial \theta}(x; \theta) \frac{\partial h_i^{(l)}(\bar{\alpha})}{\partial \theta}(\bar{x}; \theta),$$

where $\mathcal{D}_{\theta, \leq l}$ denotes the parameters up to layer l . In the infinite-channel limit, this quantity does not depend on the choice of channel i .

For $\alpha, \bar{\alpha} \in \Omega^{(l)}$, define

$$\Lambda^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) = \begin{pmatrix} \Sigma^{(l)}(x, x; \alpha, \alpha) & \Sigma^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) \\ \Sigma^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) & \Sigma^{(l)}(\bar{x}, \bar{x}; \bar{\alpha}, \bar{\alpha}) \end{pmatrix},$$

and

$$\dot{\Sigma}^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) = \mathbb{E}_{(u, v) \sim \mathcal{N}(0, \Lambda^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}))} [\phi'(u) \phi'(v)].$$

When $\phi = \text{ReLU}$, the value of $\phi'(0)$ is irrelevant for this expectation, since a non-degenerate Gaussian random variable is equal to zero with probability zero. We may therefore define $\phi'(0)$ arbitrarily. Then, for the convolutional architecture defined above, the infinite-channel NTK satisfies

$$\begin{aligned} \Theta^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \Sigma^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) \\ &= + \frac{\sigma_w^2}{|\mathcal{P}|} \sum_{\beta \in \mathcal{P}} \dot{\Sigma}^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta) \Theta^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta). \end{aligned}$$

Equivalently, the covariance and NTK recursions are coupled through

$$\begin{aligned} \Sigma^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \frac{\sigma_w^2}{|\mathcal{P}|} \sum_{\beta \in \mathcal{P}} \mathbb{E}_{(u, v) \sim \mathcal{N}(0, \Lambda^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta))} [\phi(u) \phi(v)] + \sigma_b^2, \\ \Theta^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \Sigma^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) + \frac{\sigma_w^2}{|\mathcal{P}|} \sum_{\beta \in \mathcal{P}} \dot{\Sigma}^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta) \Theta^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta). \end{aligned}$$

4.4.1 Notation and scaling

To ease notation, we omit the explicit dependence on $x, \bar{x}$ and on θ . For a fixed pair of inputs $x, \bar{x}$, we write

$$h_c^{(l)}(\alpha) := h_c^{(l)}(x; \theta)(\alpha), \quad \bar{h}_c^{(l)}(\alpha) := h_c^{(l)}(\bar{x}; \theta)(\alpha),$$

and similarly

$$x_c^{(l)}(\alpha) := x_c^{(l)}(x; \theta)(\alpha), \quad \bar{x}_c^{(l)}(\alpha) := x_c^{(l)}(\bar{x}; \theta)(\alpha).$$

We denote by $\mathcal{D}_{\theta, \leq l}$ the set of parameters up to layer l .

To match the NNGP scaling, we write the parameters in normalized form:

$$W_{c',c}^{(l)}(\beta) = \frac{\sigma_w}{\sqrt{C^{(l-1)}|\mathcal{P}|}} \omega_{c',c}^{(l)}(\beta), \quad b_{c'}^{(l)} = \sigma_b \xi_{c'}^{(l)},$$

where the normalized parameters $\omega_{c',c}^{(l)}(\beta)$ and $\xi_{c'}^{(l)}$ are independent standard Gaussian random variables. The NTK is defined with respect to these normalized parameters.

4.4.2 Induction initialization

We first compute the NTK at the first layer. Since the input $x^{(0)} = x$ is deterministic, the first pre-activation is

$$h_i^{(1)}(\alpha) = \frac{\sigma_w}{\sqrt{C^{(0)}|\mathcal{P}|}} \sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} \omega_{i,c}^{(1)}(\beta) x_c(\alpha + \beta) + \sigma_b \xi_i^{(1)}.$$

Similarly, for the second input $\bar{x}$,

$$\bar{h}_i^{(1)}(\bar{\alpha}) = \frac{\sigma_w}{\sqrt{C^{(0)}|\mathcal{P}|}} \sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} \omega_{i,c}^{(1)}(\beta) \bar{x}_c(\bar{\alpha} + \beta) + \sigma_b \xi_i^{(1)}.$$

The NTK at the first layer only contains derivatives with respect to the first-layer parameters. Therefore,

$$\Theta^{(1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} \frac{\partial h_i^{(1)}(\alpha)}{\partial \omega_{i,c}^{(1)}(\beta)} \frac{\partial \bar{h}_i^{(1)}(\bar{\alpha})}{\partial \omega_{i,c}^{(1)}(\beta)} + \frac{\partial h_i^{(1)}(\alpha)}{\partial \xi_i^{(1)}} \frac{\partial \bar{h}_i^{(1)}(\bar{\alpha})}{\partial \xi_i^{(1)}}.$$

Now,

$$\frac{\partial h_i^{(1)}(\alpha)}{\partial \omega_{i,c}^{(1)}(\beta)} = \frac{\sigma_w}{\sqrt{C^{(0)}|\mathcal{P}|}} x_c(\alpha + \beta), \quad \frac{\partial \bar{h}_i^{(1)}(\bar{\alpha})}{\partial \omega_{i,c}^{(1)}(\beta)} = \frac{\sigma_w}{\sqrt{C^{(0)}|\mathcal{P}|}} \bar{x}_c(\bar{\alpha} + \beta),$$

and

$$\frac{\partial h_i^{(1)}(\alpha)}{\partial \xi_i^{(1)}} = \frac{\partial \bar{h}_i^{(1)}(\bar{\alpha})}{\partial \xi_i^{(1)}} = \sigma_b.$$

Hence,

$$\Theta^{(1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sigma_b^2 + \frac{\sigma_w^2}{C^{(0)}|\mathcal{P}|} \sum_{c=1}^{C^{(0)}} \sum_{\beta \in \mathcal{P}} x_c(\alpha + \beta) \bar{x}_c(\bar{\alpha} + \beta).$$

Therefore,

$$\boxed{\Theta^{(1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \Sigma^{(1)}(x, \bar{x}; \alpha, \bar{\alpha})}$$

for all $\alpha, \bar{\alpha} \in \Omega^{(1)}$.

4.4.3 Splitting the NTK over layers

The set of parameters up to layer l can be decomposed layer by layer as

$$\mathcal{P}_{\leq l} = \bigcup_{r=1}^l \left(\{W_{a,c}^{(r)}(\beta)\}_{a,c,\beta} \cup \{b_a^{(r)}\}_a \right).$$

For $r \leq l$, we denote by

$$\Theta^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) = \Theta_W^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) + \Theta_b^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha})$$

the contribution to the layer- l NTK coming from the parameters of layer r . Thus, the NTK decomposes as

$$\Theta^{(l)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sum_{r=1}^l \left(\Theta_W^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) + \Theta_b^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) \right),$$

where

$$\Theta_W^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sum_{a=1}^{C^{(r)}} \sum_{c=1}^{C^{(r-1)}} \sum_{\beta \in \mathcal{P}} \frac{\partial h_i^{(l)}(x; \theta)(\alpha)}{\partial W_{a,c}^{(r)}(\beta)} \frac{\partial h_i^{(l)}(\bar{x}; \theta)(\bar{\alpha})}{\partial W_{a,c}^{(r)}(\beta)}$$

and

$$\Theta_b^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sum_{a=1}^{C^{(r)}} \frac{\partial h_i^{(l)}(x; \theta)(\alpha)}{\partial b_a^{(r)}} \frac{\partial h_i^{(l)}(\bar{x}; \theta)(\bar{\alpha})}{\partial b_a^{(r)}}.$$

This decomposition is only a rewriting of the sum over parameters: for each layer r , we separate the contribution of the convolutional weights and the contribution of the biases.

4.4.4 Forward-backward factorization

We now show, using the same trick as G. Yang [Yan20] that each layer contribution can be written as a product of a forward term and a backward term.

For $1 \leq r \leq l$, define the backward sensitivity

$$D_{i,a'}^{(l,r)}(x; \alpha, \gamma) := \frac{\partial h_i^{(l)}(x; \theta)(\alpha)}{\partial h_{a'}^{(r)}(x; \theta)(\gamma)}, \quad a' \in \{1, \dots, C^{(r)}\}, \quad \gamma \in \Omega^{(r)}.$$

This quantity measures how a perturbation of the pre-activation $h_{a'}^{(r)}(x; \theta)(\gamma)$ affects the later pre-activation $h_i^{(l)}(x; \theta)(\alpha)$.

Let us first compute the derivative with respect to a weight $W_{a,c}^{(r)}(\beta)$. By the chain rule, we must sum over all channels and all spatial positions at layer r :

$$\frac{\partial h_i^{(l)}(x; \theta)(\alpha)}{\partial W_{a,c}^{(r)}(\beta)} = \sum_{a'=1}^{C^{(r)}} \sum_{\gamma \in \Omega^{(r)}} \frac{\partial h_i^{(l)}(x; \theta)(\alpha)}{\partial h_{a'}^{(r)}(x; \theta)(\gamma)} \frac{\partial h_{a'}^{(r)}(x; \theta)(\gamma)}{\partial W_{a,c}^{(r)}(\beta)}.$$

The pre-activation at layer r is

$$h_{a'}^{(r)}(x; \theta)(\gamma) = \sum_{c'=1}^{C^{(r-1)}} \sum_{\beta' \in \mathcal{P}} W_{a',c'}^{(r)}(\beta') x_{c'}^{(r-1)}(x; \theta)(\gamma + \beta') + b_{a'}^{(r)}.$$

Since the parameter $W_{a,c}^{(r)}(\beta)$ only appears in the channel a , input channel c , and patch position β , we get

$$\frac{\partial h_{a'}^{(r)}(x; \theta)(\gamma)}{\partial W_{a,c}^{(r)}(\beta)} = \mathbf{1}_{a'=a} x_c^{(r-1)}(x; \theta)(\gamma + \beta).$$

Therefore, the sum over a' collapses, and

$$\frac{\partial h_i^{(l)}(x; \theta)(\alpha)}{\partial W_{a,c}^{(r)}(\beta)} = \sum_{\gamma \in \Omega^{(r)}} D_{i,a}^{(l,r)}(x; \alpha, \gamma) x_c^{(r-1)}(x; \theta)(\gamma + \beta).$$

Similarly, for the bias term, the chain rule gives

$$\frac{\partial h_i^{(l)}(x; \theta)(\alpha)}{\partial b_a^{(r)}} = \sum_{a'=1}^{C^{(r)}} \sum_{\gamma \in \Omega^{(r)}} \frac{\partial h_i^{(l)}(x; \theta)(\alpha)}{\partial h_{a'}^{(r)}(x; \theta)(\gamma)} \frac{\partial h_{a'}^{(r)}(x; \theta)(\gamma)}{\partial b_a^{(r)}}.$$

Since

$$\frac{\partial h_{a'}^{(r)}(x; \theta)(\gamma)}{\partial b_a^{(r)}} = \mathbf{1}_{a'=a},$$

we obtain

$$\frac{\partial h_i^{(l)}(x; \theta)(\alpha)}{\partial b_a^{(r)}} = \sum_{\gamma \in \Omega^{(r)}} D_{i,a}^{(l,r)}(x; \alpha, \gamma).$$

Substituting the weight derivative into $\Theta_W^{(l,r)}$, we obtain

$$\begin{aligned} \Theta_W^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \sum_{a=1}^{C^{(r)}} \sum_{c=1}^{C^{(r-1)}} \sum_{\beta \in \mathcal{P}} \left(\sum_{\gamma \in \Omega^{(r)}} D_{i,a}^{(l,r)}(x; \alpha, \gamma) x_c^{(r-1)}(x; \theta)(\gamma + \beta) \right) \\ &\quad \times \left(\sum_{\bar{\gamma} \in \Omega^{(r)}} D_{i,a}^{(l,r)}(\bar{x}; \bar{\alpha}, \bar{\gamma}) x_c^{(r-1)}(\bar{x}; \theta)(\bar{\gamma} + \beta) \right). \end{aligned}$$

Rearranging the sums gives

$$\begin{aligned} \Theta_W^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \sum_{a=1}^{C^{(r)}} \sum_{\gamma, \bar{\gamma} \in \Omega^{(r)}} D_{i,a}^{(l,r)}(x; \alpha, \gamma) D_{i,a}^{(l,r)}(\bar{x}; \bar{\alpha}, \bar{\gamma}) \\ &\quad \times \sum_{c=1}^{C^{(r-1)}} \sum_{\beta \in \mathcal{P}} x_c^{(r-1)}(x; \theta)(\gamma + \beta) x_c^{(r-1)}(\bar{x}; \theta)(\bar{\gamma} + \beta). \end{aligned}$$

The term

$$\sum_{c=1}^{C^{(r-1)}} \sum_{\beta \in \mathcal{P}} x_c^{(r-1)}(x; \theta)(\gamma + \beta) x_c^{(r-1)}(\bar{x}; \theta)(\bar{\gamma} + \beta)$$

is the forward contribution: it only depends on the activations entering layer r .

The term

$$D_{i,a}^{(l,r)}(x; \alpha, \gamma) D_{i,a}^{(l,r)}(\bar{x}; \bar{\alpha}, \bar{\gamma})$$

is the backward contribution: it measures how perturbations at layer r propagate to the output pre-activation at layer l .

For the bias contribution, substituting the bias derivative gives

$$\begin{aligned}\Theta_b^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \sum_{a=1}^{C(r)} \left(\sum_{\gamma \in \Omega^{(r)}} D_{i,a}^{(l,r)}(x; \alpha, \gamma) \right) \left(\sum_{\bar{\gamma} \in \Omega^{(r)}} D_{i,a}^{(l,r)}(\bar{x}; \bar{\alpha}, \bar{\gamma}) \right) \\ &= \sum_{a=1}^{C(r)} \sum_{\gamma, \bar{\gamma} \in \Omega^{(r)}} D_{i,a}^{(l,r)}(x; \alpha, \gamma) D_{i,a}^{(l,r)}(\bar{x}; \bar{\alpha}, \bar{\gamma}).\end{aligned}$$

Hence, the full contribution of layer r is

$$\begin{aligned}\Theta^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \Theta_W^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) + \Theta_b^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) \\ &= \sum_{a=1}^{C(r)} \sum_{\gamma, \bar{\gamma} \in \Omega^{(r)}} D_{i,a}^{(l,r)}(x; \alpha, \gamma) D_{i,a}^{(l,r)}(\bar{x}; \bar{\alpha}, \bar{\gamma}) \\ &\quad \times \left[1 + \sum_{c=1}^{C(r-1)} \sum_{\beta \in \mathcal{P}} x_c^{(r-1)}(x; \theta)(\gamma + \beta) x_c^{(r-1)}(\bar{x}; \theta)(\bar{\gamma} + \beta) \right].\end{aligned}$$

Thus, each layer contribution has a forward-backward structure. The forward factor depends on the activations entering layer r , while the backward factor depends on the sensitivities from layer l back to layer r .

4.4.5 Induction step

We now derive the NTK recursion step using the layer-wise NTK contributions. The goal is to find an expression of $\Theta^{(l+1)}$ depending on $\Theta^{(l)}$

From the forward-backward factorization derived above, the contribution of the parameters of layer r to the NTK at layer l can be written as

$$\Theta^{(l,r)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sum_{a=1}^{C(r)} \sum_{\gamma, \bar{\gamma} \in \Omega^{(r)}} D_{i,a}^{(l,r)}(\alpha, \gamma) \bar{D}_{i,a}^{(l,r)}(\bar{\alpha}, \bar{\gamma}) F^{(r)}(x, \bar{x}; \gamma, \bar{\gamma}),$$

where the forward factor is

$$F^{(r)}(x, \bar{x}; \gamma, \bar{\gamma}) = \sigma_b^2 + \frac{\sigma_w^2}{C^{(r-1)}|\mathcal{P}|} \sum_{c=1}^{C(r-1)} \sum_{\beta \in \mathcal{P}} x_c^{(r-1)}(\gamma + \beta) \bar{x}_c^{(r-1)}(\bar{\gamma} + \beta).$$

In the infinite-width limit,

$$F^{(r)}(x, \bar{x}; \gamma, \bar{\gamma}) \longrightarrow \Sigma^{(r)}(x, \bar{x}; \gamma, \bar{\gamma}).$$

We now study how this factorization evolves when passing from layer l to layer $l+1$.

For $\alpha \in \Omega^{(l+1)}$, the next-layer pre-activation is

$$h_i^{(l+1)}(\alpha) = \frac{\sigma_w}{\sqrt{C^{(l)}|\mathcal{P}|}} \sum_{c=1}^{C(l)} \sum_{\beta \in \mathcal{P}} \omega_{i,c}^{(l+1)}(\beta) \phi\left(h_c^{(l)}(\alpha + \beta)\right) + \sigma_b \xi_i^{(l+1)}.$$

Therefore, for $r \leq l$, the backward sensitivity from layer $l+1$ to layer r satisfies

$$D_{i,a}^{(l+1,r)}(\alpha, \gamma) = \frac{\partial h_i^{(l+1)}(\alpha)}{\partial h_a^{(r)}(\gamma)}.$$

By the chain rule through layer l ,

$$D_{i,a}^{(l+1,r)}(\alpha, \gamma) = \sum_{c=1}^{C^{(l)}} \sum_{\beta \in \mathcal{P}} \frac{\partial h_i^{(l+1)}(\alpha)}{\partial h_c^{(l)}(\alpha + \beta)} \frac{\partial h_c^{(l)}(\alpha + \beta)}{\partial h_a^{(r)}(\gamma)}.$$

Now,

$$\frac{\partial h_i^{(l+1)}(\alpha)}{\partial h_c^{(l)}(\alpha + \beta)} = \frac{\sigma_w}{\sqrt{C^{(l)}|\mathcal{P}|}} \omega_{i,c}^{(l+1)}(\beta) \phi' \left(h_c^{(l)}(\alpha + \beta) \right).$$

Hence,

$$D_{i,a}^{(l+1,r)}(\alpha, \gamma) = \frac{\sigma_w}{\sqrt{C^{(l)}|\mathcal{P}|}} \sum_{c=1}^{C^{(l)}} \sum_{\beta \in \mathcal{P}} \omega_{i,c}^{(l+1)}(\beta) \phi' \left(h_c^{(l)}(\alpha + \beta) \right) D_{c,a}^{(l,r)}(\alpha + \beta, \gamma).$$

Similarly,

$$\bar{D}_{i,a}^{(l+1,r)}(\bar{\alpha}, \bar{\gamma}) = \frac{\sigma_w}{\sqrt{C^{(l)}|\mathcal{P}|}} \sum_{\bar{c}=1}^{C^{(l)}} \sum_{\bar{\beta} \in \mathcal{P}} \omega_{i,\bar{c}}^{(l+1)}(\bar{\beta}) \phi' \left(\bar{h}_{\bar{c}}^{(l)}(\bar{\alpha} + \bar{\beta}) \right) \bar{D}_{\bar{c},a}^{(l,r)}(\bar{\alpha} + \bar{\beta}, \bar{\gamma}).$$

We now plug these expressions into the factorized form of $\Theta^{(l+1,r)}$, for $r \leq l$:

$$\Theta^{(l+1,r)}(x, \bar{x}; \alpha, \bar{\alpha}) = \sum_{a=1}^{C^{(r)}} \sum_{\gamma, \bar{\gamma} \in \Omega^{(r)}} D_{i,a}^{(l+1,r)}(\alpha, \gamma) \bar{D}_{i,a}^{(l+1,r)}(\bar{\alpha}, \bar{\gamma}) F^{(r)}(x, \bar{x}; \gamma, \bar{\gamma}).$$

Substituting the previous expressions gives

$$\begin{aligned} \Theta^{(l+1,r)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \frac{\sigma_w^2}{C^{(l)}|\mathcal{P}|} \sum_{c,\bar{c}=1}^{C^{(l)}} \sum_{\beta, \bar{\beta} \in \mathcal{P}} \omega_{i,c}^{(l+1)}(\beta) \omega_{i,\bar{c}}^{(l+1)}(\bar{\beta}) \\ &\quad \times \phi' \left(h_c^{(l)}(\alpha + \beta) \right) \phi' \left(\bar{h}_{\bar{c}}^{(l)}(\bar{\alpha} + \bar{\beta}) \right) \\ &\quad \times \sum_{a=1}^{C^{(r)}} \sum_{\gamma, \bar{\gamma} \in \Omega^{(r)}} D_{c,a}^{(l,r)}(\alpha + \beta, \gamma) \bar{D}_{\bar{c},a}^{(l,r)}(\bar{\alpha} + \bar{\beta}, \bar{\gamma}) F^{(r)}(x, \bar{x}; \gamma, \bar{\gamma}). \end{aligned}$$

The last sum is exactly the layer- r contribution to the layer- l NTK between channels c and $\bar{c}$. In the infinite-width limit, channels are independent and identically distributed, and cross-channel NTK terms vanish asymptotically. Therefore,

$$\sum_{a=1}^{C^{(r)}} \sum_{\gamma, \bar{\gamma} \in \Omega^{(r)}} D_{c,a}^{(l,r)}(\alpha + \beta, \gamma) \bar{D}_{\bar{c},a}^{(l,r)}(\bar{\alpha} + \bar{\beta}, \bar{\gamma}) F^{(r)}(x, \bar{x}; \gamma, \bar{\gamma}) \longrightarrow \mathbf{1}_{c=\bar{c}} \Theta^{(l,r)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \bar{\beta}).$$

Thus,

$$\begin{aligned} \Theta^{(l+1,r)}(x, \bar{x}; \alpha, \bar{\alpha}) &\longrightarrow \frac{\sigma_w^2}{C^{(l)}|\mathcal{P}|} \sum_{c=1}^{C^{(l)}} \sum_{\beta \in \mathcal{P}} \omega_{i,c}^{(l+1)}(\beta) \omega_{i,c}^{(l+1)}(\bar{\beta}) \\ &\quad \times \phi' \left(h_c^{(l)}(\alpha + \beta) \right) \phi' \left(\bar{h}_c^{(l)}(\bar{\alpha} + \bar{\beta}) \right) \\ &\quad \times \Theta^{(l,r)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \bar{\beta}). \end{aligned}$$

At this stage, the double sum over β and $\bar{\beta}$ is still present. The patch indices should not be collapsed before taking the infinite-width limit.

The collapse of the patch indices comes from the average over channels. Indeed, the weights $\omega_{i,c}^{(l+1)}(\beta)$ are independent across patch positions and are independent of the previous-layer quantities. Hence,

$$\mathbb{E} \left[\omega_{i,c}^{(l+1)}(\beta) \omega_{i,c}^{(l+1)}(\bar{\beta}) \right] = \mathbf{1}_{\beta=\bar{\beta}}.$$

Therefore, by the law of large numbers over the channel index c , the terms with $\beta \neq \bar{\beta}$ vanish in the infinite-width limit. Moreover,

$$\phi' \left(h_c^{(l)}(\alpha + \beta) \right) \phi' \left(\bar{h}_c^{(l)}(\bar{\alpha} + \beta) \right)$$

converges in empirical average to

$$\dot{\Sigma}^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta),$$

where

$$\dot{\Sigma}^{(l)}(x, \bar{x}; \gamma, \bar{\gamma}) := \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \lambda^{(l)}(x, \bar{x}; \gamma, \bar{\gamma}))} \left[\phi'(u) \phi'(v) \right].$$

Consequently, for $r \leq l$,

$$\Theta^{(l+1,r)}(x, \bar{x}; \alpha, \bar{\alpha}) = \frac{\sigma_w^2}{|\mathcal{P}|} \sum_{\beta \in \mathcal{P}} \dot{\Sigma}^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta) \Theta^{(l,r)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta).$$

We now sum over all previous layers $r = 1, \dots, l$. Since

$$\Theta^{(l)}(x, \bar{x}; \gamma, \bar{\gamma}) = \sum_{r=1}^l \Theta^{(l,r)}(x, \bar{x}; \gamma, \bar{\gamma}),$$

we obtain the total contribution of the old parameters:

$$\begin{aligned} \sum_{r=1}^l \Theta^{(l+1,r)}(x, \bar{x}; \alpha, \bar{\alpha}) &= \frac{\sigma_w^2}{|\mathcal{P}|} \sum_{\beta \in \mathcal{P}} \dot{\Sigma}^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta) \\ &\quad \times \sum_{r=1}^l \Theta^{(l,r)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta) \\ &= \frac{\sigma_w^2}{|\mathcal{P}|} \sum_{\beta \in \mathcal{P}} \dot{\Sigma}^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta) \Theta^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta). \end{aligned}$$

It remains to add the contribution of the new parameters of layer $l+1$. By the same calculation as for the NNGP covariance,

$$\Theta^{(l+1,l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \Sigma^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}).$$

Indeed, differentiating $h_i^{(l+1)}(\alpha)$ with respect to the new normalized parameters gives the forward covariance contribution of the layer.

Combining the old-parameter and new-parameter contributions, we finally obtain

$$\Theta^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) = \Sigma^{(l+1)}(x, \bar{x}; \alpha, \bar{\alpha}) + \frac{\sigma_w^2}{|\mathcal{P}|} \sum_{\beta \in \mathcal{P}} \dot{\Sigma}^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta) \Theta^{(l)}(x, \bar{x}; \alpha + \beta, \bar{\alpha} + \beta)$$

for all $\alpha, \bar{\alpha} \in \Omega^{(l+1)}$.

All identities above should be understood as limits of the corresponding finite-width empirical kernels, with convergence in probability as the channel widths tend to infinity sequentially. Under this limit, the random empirical NTK becomes deterministic and satisfies the stated recursion.

4.5 NTK after final readout

After the last convolutional layer, the final NTK is obtained through a global average pooling operation. Recall that the network output is defined by

$$f_i(x; \theta) = \frac{1}{|\Omega^{(L)}|} \sum_{\alpha \in \Omega^{(L)}} h_i^{(L)}(x; \theta)(\alpha).$$

Using the linearity of the global average pooling readout, for any parameter $\theta \in \mathcal{D}_{\theta, \leq L}$,

$$\frac{\partial f_i(x; \theta)}{\partial \theta} = \frac{1}{|\Omega^{(L)}|} \sum_{\alpha \in \Omega^{(L)}} \frac{\partial h_i^{(L)}(x; \theta)(\alpha)}{\partial \theta}.$$

Therefore,

$$\begin{aligned} \Theta(x, \bar{x}) &= \sum_{\theta \in \mathcal{D}_{\theta, \leq L}} \frac{\partial f_i(x; \theta)}{\partial \theta} \frac{\partial f_i(\bar{x}; \theta)}{\partial \theta} \\ &= \frac{1}{|\Omega^{(L)}|^2} \sum_{\alpha, \bar{\alpha} \in \Omega^{(L)}} \sum_{\theta \in \mathcal{D}_{\theta, \leq L}} \frac{\partial h_i^{(L)}(x; \theta)(\alpha)}{\partial \theta} \frac{\partial h_i^{(L)}(\bar{x}; \theta)(\bar{\alpha})}{\partial \theta} \\ &= \frac{1}{|\Omega^{(L)}|^2} \sum_{\alpha, \bar{\alpha} \in \Omega^{(L)}} \Theta^{(L)}(x, \bar{x}; \alpha, \bar{\alpha}). \end{aligned}$$

Finally, the NTK after global average pooling is the average of all contributions

$$\Theta(x, \bar{x}) = \frac{1}{|\Omega^{(L)}|^2} \sum_{\alpha, \bar{\alpha} \in \Omega^{(L)}} \Theta^{(L)}(x, \bar{x}; \alpha, \bar{\alpha})$$

where $\Theta^{(L)}(x, \bar{x}; \alpha, \bar{\alpha})$ is the convolutional NTK before the final readout.

5 Numerical experiments

In the following section, we compare Convolutional Neural Networks and Multi-Layer Perceptrons to their theoretical infinite-width limit. All experiments are conducted on the MNIST (??) dataset. For kernel computations, we use a finite subset of the dataset because the storage and computation of Gram matrices scale quadratically with the number of samples.

All experiments use networks of depth $L = 3$, both for the MLP and the CNN architectures. Unless stated otherwise, all hidden layers have the same width C . For CNNs, this width corresponds to the number of channels in each convolutional layer, while for MLPs it corresponds to the number of neurons in each hidden layer. We use the ReLU activation function throughout all experiments:

$$\phi(u) = \max(0, u).$$

The initialization hyperparameters are fixed to

$$\sigma_w = 1, \quad \sigma_b = 1.$$

Thus, all observed differences between MLP and CNN kernels come from the architecture and from the finite-width effects, rather than from changes in activation or initialization scale.

To compute the theoretical convolutional kernels efficiently, we exploit the structure of the patch sums appearing in the CNN covariance and NTK recursions. We use cumulative sums over the spatial dimensions to evaluate the required patch sums more efficiently than a naive repeated approach. This allows the convolutional NTK recursion to be implemented with complexity proportional to the number of pairs of spatial positions, namely $\mathcal{O}(H^2W^2)$, for images of spatial size $H \times W$, up to constants depending on the kernel size and depth.

For most experiments, we use the standard grayscale MNIST dataset with images of size 28×28 . By contrast, for some experiments involving Gram matrices, we use a low-resolution version of MNIST with images of size 8×8 . Indeed, for a dataset subset of size X , the computation of all pairwise CNN kernels scales as $\mathcal{O}(H^2W^2X^2)$, because we must compute a spatially structured kernel for every pair of input images.

5.1 Experiment 1: finite-width convergence to infinite-width kernels

The goal of this experiment is to compare empirical finite-width kernels with their infinite-width limits. We evaluate the NTK of both architectures around a fixed MNIST input and compare the finite-width empirical kernels with the corresponding theoretical kernels.

This experiment is conducted on the standard grayscale MNIST dataset, with images of size 28×28 . Both architectures have depth $L = 3$. For the CNN, we use convolutional layers with kernel size 5. For the MLP, the input image is flattened into a vector before being passed through the fully-connected layers.

5.1.1 Results

The results (figure 1) show that increasing the width makes the empirical NTK closer to its deterministic infinite-width limit. For small widths, the finite-width kernel still exhibit fluctuations. As the width increases, these fluctuations decrease and the empirical curves concentrate around the theoretical NTK.

5.1.2 Discussion

These plots should be read as qualitative examples only. They are computed around a single input sample and for only a few widths, because of limited computational resources. Therefore, they are not sufficient to prove statistically clear convergence to the infinite-width limit.

The goal of this experiment is mainly to check that the infinite-width NTK implementation behaves as expected. As the width increases, the empirical NTK gets closer to the theoretical curve, which is consistent with the NTK theory. This mainly echoes the type of comparison shown in the original NTK paper [?].

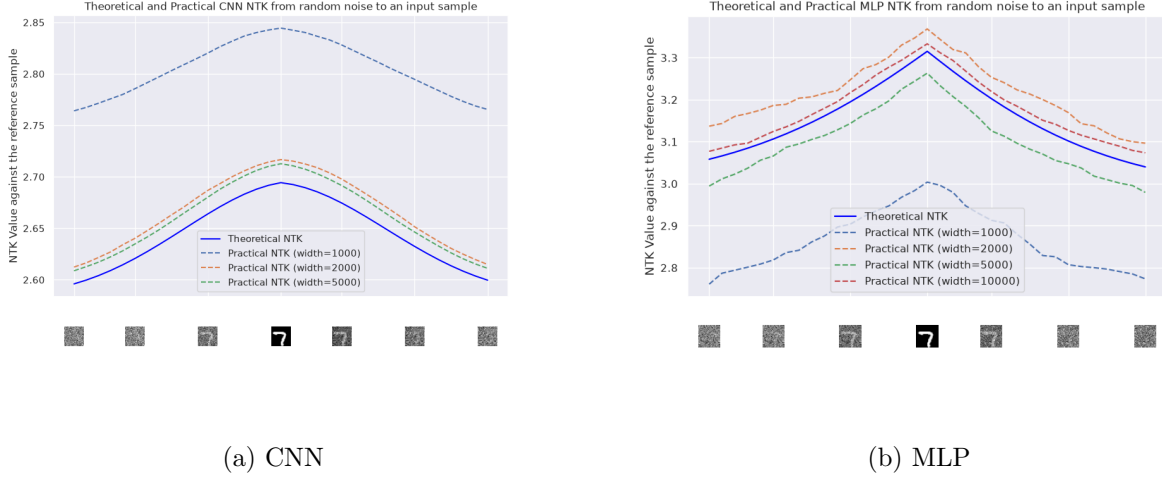


Figure 1: Example of convergence of the NTK to a deterministic limit for a finite set of widths

We also observe that the theoretical CNN NTK curve is more rounded around the input sample than the MLP one. This suggests that, in this experiment, the CNN NTK is less sensitive to noise.

5.2 Experiment 2: NTK drift during training

For a finite-width kernel K_m and infinite-width kernel K_∞ , we measure the relative Frobenius error

$$\text{RelErr}(K_\theta, K_{\theta'}) = \frac{\|K_\theta - K_{\theta'}\|_F}{\|K_{\theta'}\|_F}.$$

The goal of this experiment is to measure how much the empirical NTK changes during training. Let Θ_t denote the empirical NTK at training step t . We define the NTK drift as

$$D_t = \frac{\|\Theta_t - \Theta_0\|_F}{\|\Theta_0\|_F}.$$

In this experiment, the networks are trained with the mean squared error loss, using one-hot encoded MNIST labels. We use the Adam optimizer with learning rate 10^{-3} . The empirical NTK is computed every 10 training steps.

Because computing NTK Gram matrices is expensive, we compute them on a small subset of 10 samples taken in the training dataset. Thus, each empirical NTK is represented by a 10×10 Gram matrix.

5.2.1 Results

Figure 3 shows the NTK drift during training for several widths. For both architectures, the drift increases at the beginning of training and then tends to stabilize. We also observe that larger widths generally lead to smaller NTK drift. This is consistent with the idea that wider networks stay closer to their initialization kernel.

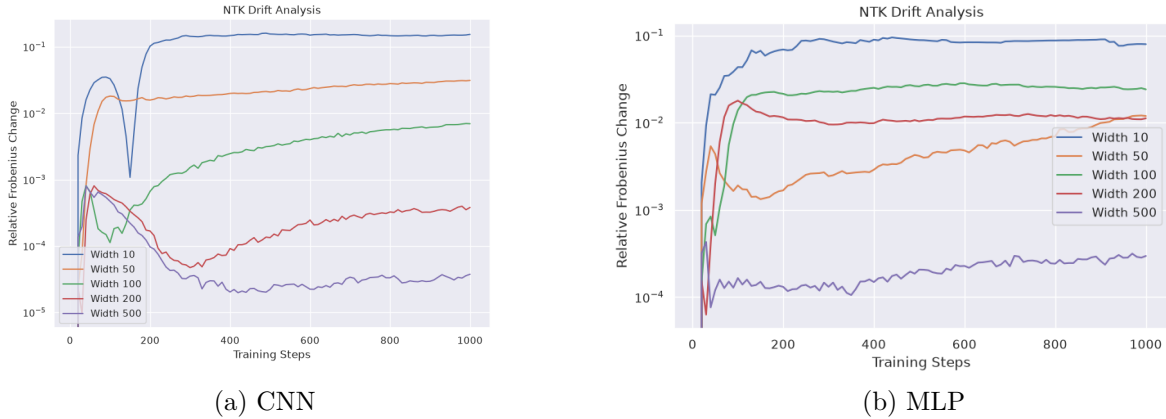


Figure 2: NTK drift across architectures. Each curve reports the relative Frobenius distance between the empirical NTK Gram matrix at a fixed width and its initial value.

5.2.2 Discussion

This experiment is aiming to show that in our setup, an over-parametrized network is following the lazy training regime, introduced by [COB19]. In this regime, the network parameters move only slightly during training, so the empirical NTK remains close to its initial value. While this regime holds, we can study the finite-model behavior with the NTK tools. Training can then be approximated by kernel gradient descent with a fixed kernel, the NTK.

Our results are coherent with this picture: wider networks tend to have smaller NTK drift. This suggests that increasing the width makes the training dynamics closer to the infinite-width NTK regime. Conversely, a larger drift means that the kernel changes more during training, which may indicate stronger feature learning beyond the lazy regime.

In these experiments, the global behavior is quite similar for CNNs and MLPs. For both architectures, the drift stabilizes during training and decreases when the width increases.

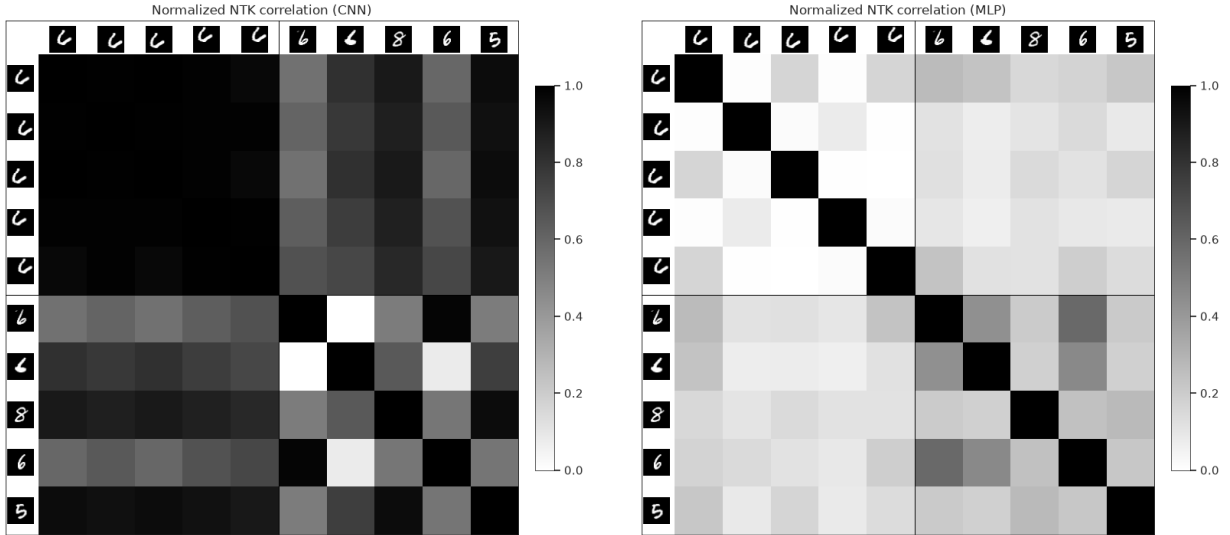
5.3 Experiment 3: Translation invariance properties of CNN

This experiment is a small qualitative test of the behavior of the CNN NTK under translations. We use a subset of 10 images. Among them, the first images are translated versions of the same MNIST digit, while the remaining images are different MNIST samples used as references. The goal is not to prove translation invariance, but to check whether the CNN NTK shows a more translation-stable behavior than the MLP NTK.

The translated images are obtained by shifting the reference image by 4 pixels in different directions. We then compute the NTK Gram matrix on this subset and normalize it into a correlation matrix:

$$\text{Corr}(i, j) = \frac{\Theta(x_i, x_j)}{\sqrt{\Theta(x_i, x_i)\Theta(x_j, x_j)}}.$$

This normalization removes differences in kernel magnitude and makes the comparison focus on similarity between inputs.



(a) CNN

(b) MLP

Figure 3: NTK values for small translations of the same input image (top left corner), and other inputs for reference (bottom right corner).

5.3.1 Discussion

Although the CNN used here is not exactly translation invariant, the result shows a clear translation-stable behavior. The translated versions of the same digit remain strongly correlated for the CNN NTK. This is much less visible for the MLP NTK, which is closer to a diagonal matrix. This suggests that the convolutional structure helps preserve similarity under small translations.

However, the CNN NTK is not only grouping identical digits. For example, some different digits, such as an 8 and a 5, can still have a large correlation. This shows that the kernel similarity is not simply equivalent to the class label. The effect is also not perfectly consistent. In particular, two examples of the digit 6 appear far apart for the CNN NTK, while they are considered closer by the MLP NTK. Thus, this experiment should be read as a qualitative illustration of CNN-specific behavior, rather than as a complete analysis of translation invariance.

5.4 Experiment 4: spectral comparison of Gram matrices

The goal of this experiment is to compare the geometry induced by MLP and CNN kernels on MNIST. Given a Gram matrix K , we study its eigenvalues

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0.$$

and other properties linked to their distributions like the effective rank :

$$r_{\text{eff}}(K) = \frac{(\sum_{i=1}^n \lambda_i)^2}{\sum_{i=1}^n \lambda_i^2}.$$

To better study the eigenvalues, it is important that we also consider the centered Gram matrices. It serve the purpose of removing the constant component of those matrices. Given the

centering matrix :

$$H = I_n - \frac{1}{n} \mathbf{1} \mathbf{1}^\top,$$

the centered kernel matrix is defined as :

$$K_c = H K H.$$

Centering removes the common direction shared by all samples, corresponding to the constant vector $\mathbf{1}$. This is useful because raw NTK Gram matrices can have a large constant component, which may dominate the first eigenvalue and hide the finer geometry of the data.

The experiment is computed on a low-resolution version of MNIST, where images are resized to 8×8 . This reduction is used because Gram matrix computations are expensive for spatial kernels. For the CNN, we use kernel size 3. The initialization parameters are fixed to

$$\sigma_w = 1, \quad \sigma_b = 0.1$$

5.4.1 Results

Model	Kernel	$\lambda_{\max}$	Condition no.	Effective rank	Top energy share	Const. align.
CNN	Raw	21.117	1207.58	3.71	0.5029	0.9919
CNN	Centered	3.429	7.52×10^7	14.94	—	—
MLP	Raw	13.346	205.78	8.65	0.3239	0.9835
MLP	Centered	2.072	5.86×10^6	41.33	—	—

Table 1: Spectral properties of the CNN and MLP NTK Gram matrices. The centered condition number is inflated by the near-zero eigenvalue introduced by kernel centering.

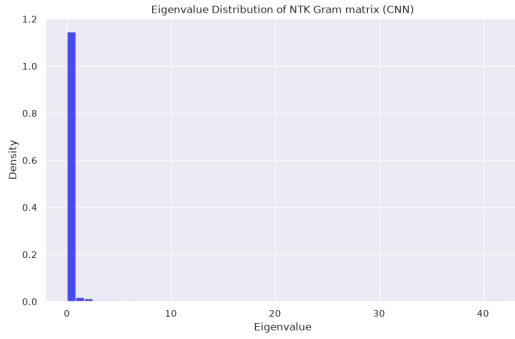
Table 1 and Figure 4 show that the eigenvalues are very unevenly distributed. For both architectures, most eigenvalues are close to zero, while a small number of directions contain most of the energy. This effect is especially visible for the raw CNN NTK, which has a lower effective rank and a larger top energy share than the MLP NTK.

5.4.2 Discussion

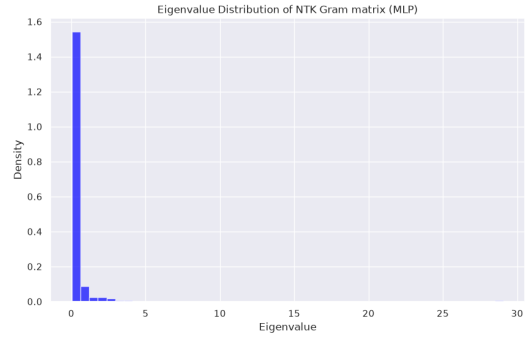
The CNN NTK appears to have a more concentrated spectrum than the MLP NTK. Its effective rank is lower, and around half of the energy is contained in the largest eigenvalue. This suggests that, in this setup, the CNN kernel separates the MNIST samples less strongly than the MLP kernel.

One possible explanation is the global average pooling readout. By averaging over all spatial positions, GAP reduces spatial information before the final kernel is obtained. This can be useful for some invariance properties, but it may also reduce the ability of the kernel to distinguish images that differ by localized details.

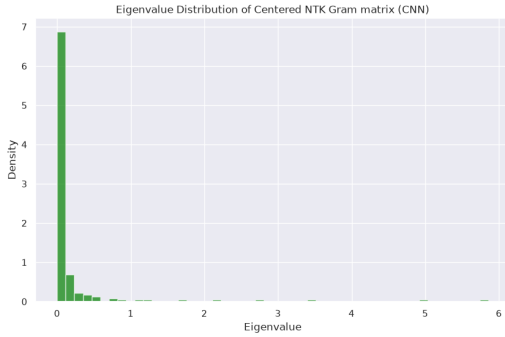
The large first eigenvalue in the raw kernels may also come from the structure of MNIST itself. MNIST images are not arbitrary images: they lie on a specific low-dimensional data manifold, with many shared properties such as background, centering, and stroke structure. This can naturally create a strong common direction in the Gram matrix.



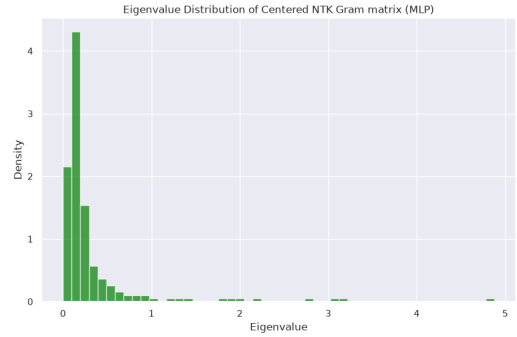
(a) CNN NTK Eigenvalues distribution



(b) MLP NTK Eigenvalues distribution



(c) Centered CNN NTK Eigenvalues distribution



(d) Centered MLP NTK Eigenvalues distribution

Figure 4: NTK Gram matrix eigenvalues distribution, for CNN and MLP architecture. Centered matrices have been stripped of their common direction $(1, 1, \dots, 1)$ to all entries of the dataset.

After centering, the effective rank increases for both architectures. This confirms that a large part of the raw spectrum comes from a common component shared by all samples. The MLP centered kernel still has a larger effective rank than the CNN centered kernel, which suggests that the two architectures organize the dataset differently.

5.5 Experiment 5: Classes discrimination ability

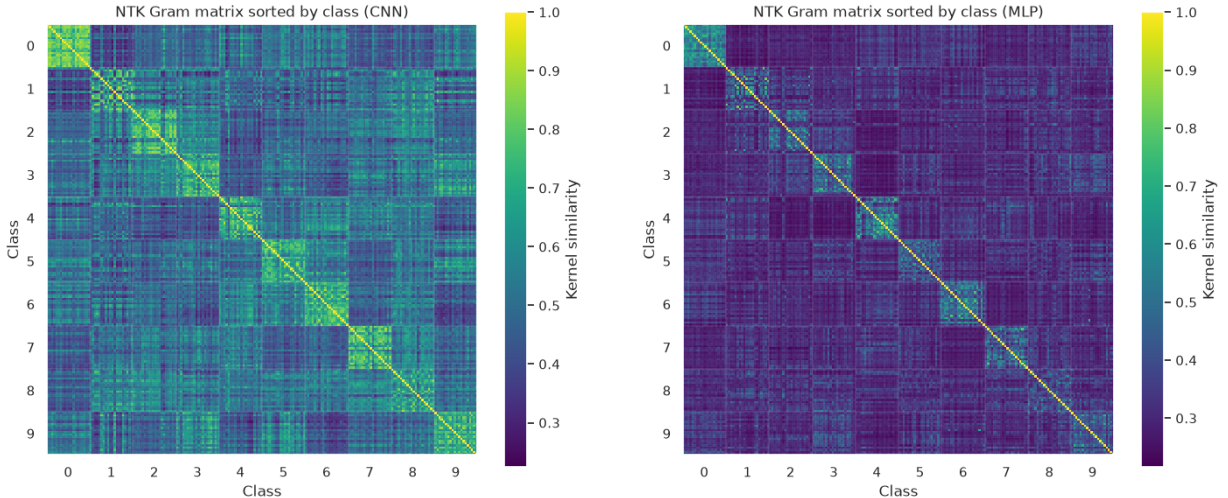
The goal of this experiment is to test whether the kernels already contain class information before training. More precisely, we ask whether samples from the same MNIST class have larger NTK similarity than samples from different classes. This gives a simple way to compare the class-discrimination ability induced by the architecture itself, independently of training. Since CNNs are designed for image data, we expect the CNN NTK to show a clearer class structure than the MLP NTK.

5.5.1 Results

Figure 5 shows the NTK Gram matrices after sorting the samples by class. The effect is not very strong, but some diagonal block structure can be observed. This suggests that samples from the same class tend to have slightly larger kernel similarity. The CNN and MLP both show this behavior, although the difference between them is not very pronounced.

Model	Kernel	Within mean	Between mean	Gap	Relative gap
CNN	Raw	0.6952	0.4946	0.2007	0.2886
CNN	Centered	0.3714	-0.0439	0.4153	1.1183
MLP	Raw	0.4402	0.3072	0.1330	0.3022
MLP	Centered	0.1757	-0.0222	0.1979	1.1264

Table 2: Within-class and between-class NTK similarities for CNN and MLP kernels. The gap is defined as the difference between the within-class mean and the between-class mean.



(a) Sorted NTK Gram matrix (CNN)

(b) Sorted NTK Gram matrix (MLP)

Figure 5: NTK Gram matrices, with rows and columns sorted by class, each evaluated on 200 MNIST samples (20 per class).

5.5.2 Discussion

The diagonal blocks correspond to groups of samples belonging to the same class. Their presence suggests that the architecture already separates some MNIST classes before training. In other words, the NTK is not random with respect to the labels: it already induces a geometry where same-class examples are somewhat closer.

However, the effect is weaker than one might hope. The blocks are visible but not sharply separated. This agrees with Table 2: the within-class similarity is larger than the between-class similarity for both architectures, but the gap remains moderate.

We do not observe the strong difference between classes we were hoping to find on for the CNN NTK, the relative gap remains low especially for its raw Gram matrix. But we do observe general differences between classes.

These differences matter because the NTK controls the first-order training dynamics. If samples from the same class are already aligned in the kernel, then the early parameter updates are biased toward class-consistent directions. Thus, this experiment gives a small indication of how architecture can influence learning even before any training has occurred.

6 Discussion and Limitations

6.1 Discussion

The experiments suggest that infinite-width kernels provide a useful approximation for studying finite-width neural networks, but also that they do not capture all aspects of finite-width behavior. In the convergence experiment, wider empirical NTKs become closer to their infinite-width limits. Similarly, in the drift experiment, wider networks tend to have a smaller NTK drift during training. This is consistent with the NTK regime, where training is well approximated by kernel gradient descent with a nearly fixed kernel.

The comparison between CNNs and MLPs also shows that the architecture has a visible effect on the induced kernel geometry. The MLP treats the image as a flattened vector, while the CNN encodes locality and weight sharing. This difference appears most clearly in the translation experiment, where the CNN NTK preserves more similarity between translated versions of the same image than the MLP NTK.

However, the CNN kernel is not uniformly better in all the experiments. In the spectral comparison, the CNN NTK has a more concentrated spectrum and a lower effective rank than the MLP NTK. This suggests that, in this setup, the CNN kernel may distinguish individual MNIST samples less strongly. One possible reason is the global average pooling readout, which creates more invariance but also reduces spatial discrimination.

The class-sorted Gram matrices show that both architectures already contain some class-related structure before training. Samples from the same class tend to have slightly higher kernel similarity, but the effect remains moderate. Thus, the architecture influences the geometry seen by gradient descent, but the observed class separation is not strong enough to fully explain learning by itself.

Overall, this study should be understood as a controlled comparison rather than as a new theoretical framework. The goal is to connect the theoretical NTK recursions with concrete finite-width experiments, and to observe how architectural choices affect the resulting kernels.

6.2 Limitations

The main limitation of this work is computational. Computing empirical NTKs and Gram matrices is expensive, especially for convolutional architectures. As a result, several experiments are performed on small subsets of MNIST, and some results should be read as qualitative illustrations rather than statistically strong conclusions.

The experiments are also limited to MNIST. This dataset is useful for controlled experiments, but it is simple, centered, and low-dimensional compared to more realistic image datasets. Therefore, the observations made here may not directly transfer to harder datasets such as CIFAR-10 or ImageNet. Another limitation is that most experiments use only a small number of widths and initializations. A more complete study would average over several random seeds, use larger subsets of data, and report confidence intervals. This would make the convergence and drift observations more statistically reliable.

Finally, the CNN architecture used in this work is also deliberately simple. On the one hand, it was chosen because it is easier to derive and implement theoretically. On the other hand, it is

not accurate to represent modern practical CNNs. In particular, one could include local pooling layers between convolutional layers, or use more realistic readout mechanisms, to obtain kernels that may better reflect practical convolutional networks.

7 Conclusion

In this work, we studied the neural tangent kernels of MLPs and CNNs, with the goal of comparing their infinite-width limits and their finite-width behavior. The objective was not to introduce a new theoretical framework, but to build a controlled and self-contained comparison between two standard architectures.

On the theoretical side, we recalled the known NTK recursion for fully-connected networks and derived the corresponding covariance and NTK recursions for a simple convolutional architecture without padding.

On the experimental side, the results show that architecture has a strong influence on the symmetries of a neural network, on its neural tangent kernel, and therefore on its training dynamics. In particular, the CNN NTK preserves similarity under small translations more clearly than the MLP NTK, reflecting the locality and weight sharing of convolutional layers. The spectral and class-sorted Gram matrix experiments also show that the two architectures organize MNIST differently before training. Thus reaffirming that in the lazy-training regime, the choice of architecture and initialization determines in itself the geometry followed by gradient descent.

These experiments remain limited by computational cost, small dataset subsets, and the simplicity of the architectures considered. Nevertheless, they illustrate how NTK tools can be used to study architectural biases before and during training. Future work could extend this comparison to larger datasets, more random seeds, and more realistic convolutional architectures, including pooling layers and different readout choices.

References

- [COB19] Lénaïc Chizat, Edouard Oyallon, and Francis Bach. On lazy training in differentiable programming. In *Advances in Neural Information Processing Systems*, 2019.
- [JGH20] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks, 2020.
- [LBN⁺18] Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. Deep neural networks as gaussian processes. In *International Conference on Learning Representations*, 2018.
- [LXS⁺19] Jaehoon Lee, Lechao Xiao, Samuel S. Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide neural networks of any depth evolve as linear models under gradient descent. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [MRH⁺18] Alexander G. de G. Matthews, Mark Rowland, Jiri Hron, Richard E. Turner, and Zoubin Ghahramani. Gaussian process behaviour in wide deep neural networks. In *International Conference on Learning Representations*, 2018.

- [Nea96] Radford M. Neal. *Bayesian Learning for Neural Networks*, volume 118 of *Lecture Notes in Statistics*. Springer, 1996.
- [Yan20] Greg Yang. Tensor programs ii: Neural tangent kernel for any architecture, 2020.